Streaming Twitter Analytics with Kafka, Spark, and GCP

Angel Coronel
CSc 84030 Big Data Analytics
CUNY Graduate Center
December 30, 2025

***Abstract*—This project shows how a real time analytics pipeline is designed, implemented and evaluated for processing large scale data from social media, namely twitter (now rebranded as X). The system is built using various tools like Apache Kafka for ingesting data, Apache Spark for processing streaming data and Parquet to store the results after being processed. The pipeline collects mentioned entities, selected individuals and organizations, from the stream of tweets and makes a judgement using a sentiment analysis AI (artificial intelligent) tool to weigh relationships between the entities. This is all for the goal to show how modern big data analytics systems can be used to transform text streams with high volume into something usable like a constantly structured and updated results. This system was deployed in Google Cloud Platform, GCP using containers and evaluated using streaming workloads. The experiments done with this project show that the pipeline is able to process data reliably with stable resource usage, low latency and fault tolerance.**

I. Introduction

Social media platforms are constantly generating a ton of information/content from users interacting with the platform which only grows at an unpredictable rate regardless of what time or day it is. This data has meaningful insights about what people think or opinionate, about relationships between people and organizations and new trends. Still, the unbounded speed of the data makes it hard for a person or even traditional data processing techniques to analyze anything. Instead, we can process the data in batches which give a fixed window to make analysis of the data but batches can add delays and limit the "real time" changes as they are happening.

To take on this limitation, streaming data analytics systems will continuously process the data as it arrives and keep an updated state some where else. This state is defined relative to the data itself and what the analysis of the data is trying to measure or capture. Instead of doing the same computations over and over to arrive at the same results like in a static dataset, streaming systems will update the analytics outputs very close to the time the data is streamed, "near real time". This matters because relevance depends on timeliness for applications that want to have evolving relationships or an understanding in online talks.

The motivation for this project is to see how modern big data analytics frameworks can be used to build a streaming pipeline that feeds off of live social media data. The original idea was that instead of just classifying text or counting keywords to find what is relevant or trending, the project tries to form relationships between edges of what's inside the text to map certain celebrities and companies to each other based on what people are saying. The context of what people are saying depends on an AI tool that acts as a "judge" marking a tweet as positive, neutral or negative that gets added to a relationships graph with the weight being what was judged by the AI. When a specific person or organization get lumped together, the system makes a dynamic relationship graph that shows how these people and organization are being discussed in relation to one another over time within the tweets that are being streamed.

From a systems point of view, this project can be seen as a case study in designing, implementing and then evaluating a fault tolerant streaming analytics pipeline. It focuses on using big data tech like Kafka and Spark to show engineering tradeoffs related to throughput, latency, resource usage, and reliability. The project matches up with big data analytics by showing how unstructured and high volume streams can be changed into structured data for analysis.

II. System Architecture

The system is made with loosely coupled components responsible for different parts of processing data as an end-to-end streaming analytics pipeline. Building this architecture in modules was made to handle the high volume input data, do the continuous analytics and store the results so there wouldn't be major changes if at any point in the pipeline something went down. This is to account for testing the system easier, keeping maintenance relatively easier and later scalability during the tests. The design is mostly good intuition like separating ingestion and processing, not passing faults or errors between components and have durable results.

At the start, a data ingestion component has to collect the tweets and publish them to an Apache Kafka topic. The origin source is either a file that stores the tweets (in csv or json format) or using the Twitter API; the Twitter API ended up being too expensive to use for this project so all of the tests are using a file. The main input source for all the results is using Sentiment140 [1] dataset with 1.6 million tweets from Kaggle. This dataset is used as a continuous stream to stress test the system before running out of tweets to simulate continuous, flowing data through the pipeline. With the tweets in the Kafka topic, Kafka acts like a distributed commit log that holds incoming data and splits data producers from consumers. This lets the ingestion process work independently from the

analytics pipeline and gives the ability to replay data in case something goes down in processing or "consuming" the data later. Kafka also gives some guarantees that the order of the data will not be lost within partitions which is good to have for consistent processing.

Processing the stream is done by Apache Spark Structured Streaming. It consumes data straight from Kafka. Spark is the core of the system since it carries out a long streaming query that continuously will process the incoming tweets from Kafka. Spark's Structured Streaming has another level of abstraction for processing the stream that takes care of batch and streaming semantics. It'll basically allow the system to relational transformations while keeping the state of the relationship graph through the micro batches. The relationships are weighted based on Vader AI, an AI sentiment analysis tool, judging the text of a tweet that gets stored as part of the results and updates the state of the current graph. Checkpoints are used to make sure the progress and intermediate states persists throughout failures; so, duplicated data or data loss in general don't happen.

After the data has been processed through Spark, the results are written to a Parquet file that will remain inside the VM storage and can be retrieved should the system itself go down or error out. The Parquet files is a column oriented representation of the data meant for analytical work and can be queried with batch processing tools. By having the storage be separate from the processing, the system can have older data that can be analyzed independently away from the live pipeline. This way more analytics can be done "offline" without messing up with any of the real time processing.

All of the components in the system are deployed using containers with Docker. Kafka, Spark master and Spark worker run in separate containers within the VM on GCP. The storage and the VM are all set to be in the same zone (us-central1-b) to reduce communication traffic. This simplified configuring the whole project while also making it easier to monitor each component separately. This architecture works as the foundation to evaluate the performance, resource use and fault tolerance of a realistic big data analytics system.

III. Data Model

The data model is made to support to stream of unstructured text to be used later. The pipeline works on single tweet records as they come in and builds up on each one throughout processing. The results are simple but can give a lot of interpretations from the input data.

Each incoming tweet is treated as single record with the raw text, a timestamp and basic metadata. The raw text is mainly what is used throughout the rest of the pipeline. Instead of storing all of the metadata, the data model focuses only on what is relevant for the sentiment analysis in order to lower how much storage space is used and processing complexity.

```
normalized = {
    "id": raw_tweet.get("id"),
    "created_at": raw_tweet.get("created_at"),
    "text": raw_tweet.get("text"),
    "author_id": raw_tweet.get("author_id"),
    "in_reply_to_user_id": None,
    "referenced_tweets": raw_tweet.get("referenced_tweets", [
    "mentions": mentions,
    "raw_entities": {
        "hashtags": hashtags,
        "urls": urls,
    },
    "source": "twitter_api",
}
return normalized
```

Fig. 1. Logical representation of an incoming tweet record used in the pipeline.

During processing, the tweets records are added with entities from a predefined list of interest. An entity is counted as present in a tweet if their name or "pattern" (an associated identifier) is in the raw text of the tweet. When there are more than one entity in the tweet, the tweet is marked as having co-mention between those entities and a relationship is formed.

```
figures:
  # US celebrities / billionaire artists
  - canonical_name: "Taylor Swift"
    patterns: ["taylor swift", "@taylorswift13"]
    type: "person"
    country: "US"
    tags: ["celebrity", "musician", "billionaire"]

  - canonical_name: "Jay-Z"
    patterns: ["jay-z", "jay z", "@sc"]
    type: "person"
    country: "US"
    tags: ["celebrity", "musician", "billionaire", "entr

  - canonical_name: "Rihanna"
    patterns: ["rihanna", "@rihanna"]
    type: "person"
    country: "Barbados"
    tags: ["celebrity", "musician", "billionaire", "entr

  # US tech / finance billionaires
  - canonical_name: "Elon Musk"
    patterns: ["elon musk", "@elonmusk"]
    type: "person"
    country: "US"
    tags: ["billionaire", "tech", "ceo"]
```

Fig. 2. Example of entity extraction and co-mention detection within a tweet.

Sentiment info is made for each tweet using a sentiment scoring approach with Vader AI. The score represents the overall polarity of the text in a given tweet and is used to associate tall of the co-mentioned entity pairs within that tweet. This makes it so that the system can give weighted

contributions to a relationship graph and keep track of the overall sentiment of the discussion between the nodes.

```
>>> df = spark.read.parquet("/data/aggregated/edges")
df.printSchema()
>>> df.printSchema()
root
 |-- src: string (nullable = true)
 |-- dst: string (nullable = true)
 |-- count: long (nullable = true)
 |-- avg_sentiment: double (nullable = true)
 |-- stddev_sentiment: double (nullable = true)
 |-- first_seen: string (nullable = true)
 |-- last_seen: string (nullable = true)
```

Fig. 3. Sentiment scoring and weighting applied to entity relationships.

The output of the pipeline is a structured dataset where each record represents a relationship between two entities over time. Each record has the entities involved (source and destination), the sentiment analysis between them and a count of how many time people spoke about them together. The records are written into the Parquet files as persistent storage which can be batch queried or do analysis off of them.

```
+----------+----------+-----------+--------------------+
|src       |dst       |co_mentions|avg_sentiment       |
+----------+----------+-----------+--------------------+
|Elon Musk |Tesla     |64         |0.026167187499999998|
|Alibaba   |Jack Ma   |61         |0.0                 |
|SpaceX    |Tesla     |58         |0.0                 |
|Elon Musk |SpaceX    |58         |0.0                 |
|Elon Musk |Jeff Bezos|17         |0.067376470588235529|
|Bill Gates|Jeff Bezos|11         |0.09272727272727273 |
|Apple     |Tim Cook  |11         |0.26090909090909087 |
|Bill Gates|Jack Ma   |9          |0.17556666666666665 |
|Bill Gates|Elon Musk |7          |0.1931142857142857  |
|Jack Ma   |Tim Cook  |7          |0.5356142857142857  |
+----------+----------+-----------+--------------------+
only showing top 10 rows

>>>
>>> edge_stats.filter(F.col("co_mentions") >= 2) \
...     .orderBy(F.desc("avg_sentiment")) \
...     .show(10, truncate=False)
```

Fig. 4. Edge weight records.

This data model gives a mapping from unstructured text input to structured relational output. It supports both real time monitoring and offline evaluation of the entity relationships that were created.

IV. Pipeline

A. Ingestion

The ingestion pipeline makes sure that raw tweets are collected and published to Kafka. Kafka is the entry point into the system and draws the line between data production and stream processing. Zookeeper and Kafka are deployed using Docker which makes the ingestion component work and be separate from everything else.

A producer (code writing with Python) is used to generate and send messaged to a Kafka topic, tweets. The producer reads them from either a sample dataset stored locally in the VM or from Twitter's API. Both are implemented but, as previously discussed, only the sample dataset is used for performing stress tests. The tweets that are coming in are normalized to a consistent format before being sent to Kafka (covered in Data Model section). Normalizing is an important step to make sure any additional parsing can be done.

Data that goes through the ingestion component is validated using Kafka's command line interface, CLI, tools. The tools let us see what is going on inside the container, what the data looks like, any errors during implementation or testing, and check that everything is going according to plan throughout the whole project. Making sure everything goes right with ingestion ensures that Kafka is getting the continuous data and we are reliably consuming messages in real time processing.

B. Streaming

The streaming pipeline is the main part of this project. Using Spark, its responsible for changing the raw tweets to a structured graph. The tweets are read from a Kafka topic using Spark's native integration and processed as distinct events based on Kafka's partition order. Spark uses micro batch model where new data is grouped into small batches and progressively gets processed. This keeps a low processing latency.

For each micro batch, the text is parsed and normalized before getting the entities they are related to. When multiple entities appear in one tweet, all of the pairs are generated with it. The sentiment analysis is applied after this and the score of each pair is stored as part of the results. Additions keep track of how many times a pair is made and how sentiment changes over time with Spark keeping the state throughout the batches.

The aggregated relationship records are written to the Parquet files as the stream progresses. These records can be seen later offline of while the pipeline is running.

V. Experimental Setup

To run the experiments, the system was used in a controlled environment using GCP to show what a realistic deployment looks like. All of the components were deployed using a single VM with containers. This made it possible to have consistent runs while seeing what changed or remained the same under a timed load. The software used Kafka and Spark containers running the data producer and consumer in two separate windows of the same VM. Using the CLI, we could see the results during and afterwards from either of the windows or a third window to check on the first two. Checkpointing allowed the system to have persistent data in case of errors and see what happened last if there was a crash or continue processing from a given point in time if desired.

Data was generated and ingested continuously over timed intervals in order to check the performance of the pipeline. Using Kaggle's Sentiment140 dataset, we were able to simulate what a stream under large volume would look like and mark the results of the system. During a run, multiple metrics were collected including Spark's streaming logs, Kafka's offset info and the resources the container would use for a run.

How resources were being used was monitored over all of the containers running at a time to track how the processor and memory is being impacted by each container. This allowed us to see how each container uses up resources in the VM and potentially places where the system could be improved further.

This setup is how the system was tested for throughput, latency, resource efficiency and fault tolerance. By running the continuous data, stopping before the data ran out, on the pipeline, we can see the system metrics and the graph results from the tweets. The experiments are meant to test if the system can handle reliable real time analytics on twitter's streaming data.

VI. Performance Evaluation

The performance was evaluated by stress testing the system with data ingestion and continuous processing. The focus is on how effective the system processes the data, how quickly the results are done, how resources are used and how reliable to pipeline is in long runs. The goal isn't to optimize a single metric, only to show the system is predictable and robust with a (simulated) real time analytics pipeline.

A. Streaming versus Batch Processing

A main reason to use streaming architecture is the limit of batch processing for unbounded data sources. With batches, data has to first be placed into a fixed dataset before anything can be done with it which makes a system slowed and needs recomputed values repeatedly with newer data. On the other hand, the streaming pipeline handles each tweet as it comes in and updates the relationship graph as time goes on.

The system shows how streaming processing can produce dynamic results without having to reprocess historic data. The relationships evolve as new tweets arrive which reflect changes in the discussion in real time. This would be hard only using batch processing and makes the system scalable to the size of the data.

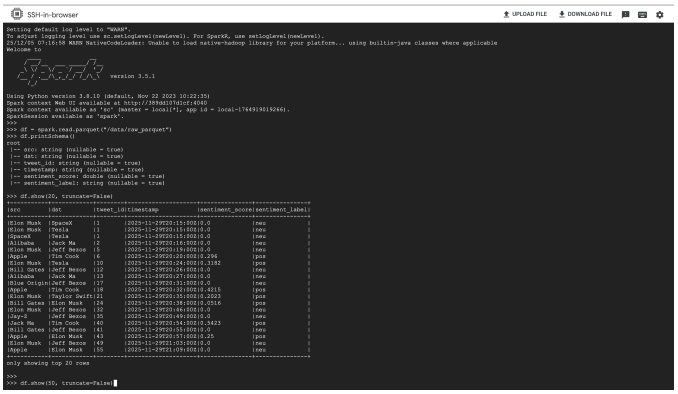


Fig. 5. Parquet file records.

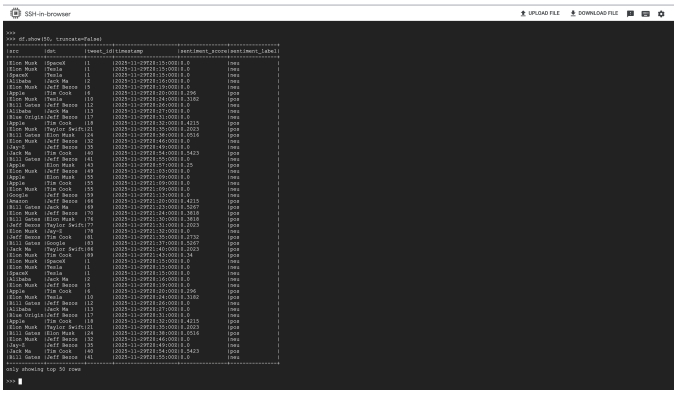


Fig. 6. Parquet records of top 50 edges.

B. Resource Usage

How resources are used was monitored while running the system to see how things change while sustaining a load of tweets. Table I summarizes CPU and memory usage across the Python ingestion baseline and the core containerized components of the system before, during and after active streaming. The baseline does not include Spark or Kafka, only parsing through tweets one at a time and adding it to a table in the end after using Vader for sentiment analysis.

TABLE I
Merged Resource Usage Summary

Component	Phase	CPU	Memory	Notes
Python baseline	Run	∼102%	∼109 MB	Stable
spark-worker	Before	0.16%	403.8 MiB	Idle baseline
	During	221.32%	1.304 GiB	Peak compute during backlog drain
	After	156.48%	1.328 GiB	CPU drops, memory plateaus
spark-master	Before	0.20%	298.9 MiB	Idle baseline
	During	129.00%	1.233 GiB	Driver + scheduling overhead
	After	97.79%	1.247 GiB	Stabilized
kafka	Before	1.11%	1022 MiB	Idle broker
	During	14.14%	1024 MiB	Moderate I/O pressure
	After	8.48%	1024 MiB	Returns near baseline
zookeeper	All	∼0.1%	120.5 MiB	Negligible impact

During the run, Spark workers used up all of the CPU cores while the memory went up a but manageable at 9% of available memory. Kafka and Zookeeper used very few resources in a given run. Once a run was over, the CPU went back down while the memory stayed relatively the same. This is a good indication that there aren't any memory leaks in the system and resources are only used while active streams are being processed.

C. Throughput and Latency

The throughput was measured by taking the rate of consumed messages from Kafka and produced relationship records. While running, Kafka offsets increased linearly without skipping any new tweet or duplicating older tweets. Spark

was able to process the micro batches keeping up with the input rate.

Latency was evaluated through Spark;s metrics and prolonged processing delays. Checking during excecution times of micro batches during the experiments was how we kept track of metrics. The system was able to keep up with input with a low end to end delay.

TABLE II
STREAMING VERSUS BATCH PROCESSING CHARACTERISTICS

Mode	Latency	Throughput	Incremental	Scalability
Streaming	Low	Moderate–High	Yes	High
Batch (Python)	High	High per job	No	Low

To do a stress test, we ran the system for 25 minutes and processed over a million tweets. THe pipeline did about 3,200 tweets per second while keeping up with a throughput up to 4,700 rows per second. Even when the micro batches lasted longer than 20 second intervals, the Kafka consumer didn't have a lag and still catched the next set of batches. Resources scaled as one would expect. Spark had a high CPU use and memory stayed steady. There were not failures, errors or crashed while stress testing and the Parquet files space grew linearly to the throughput of the run. This showed that the system could handle high throughput workloads.

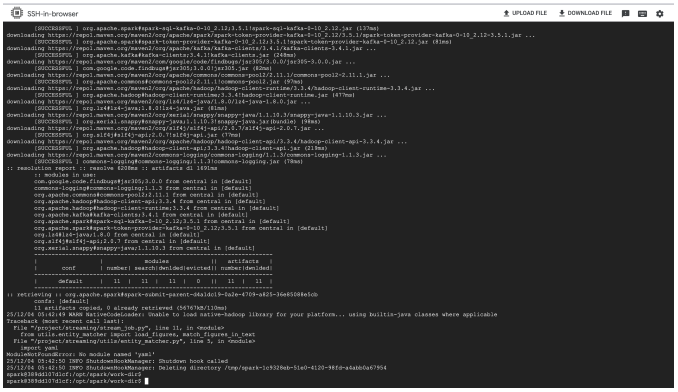


Fig. 7. Spark streaming tweets.

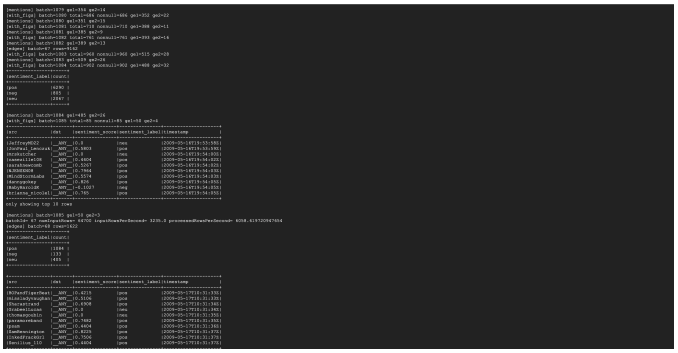


Fig. 8. Results with over a million tweets being streamed.

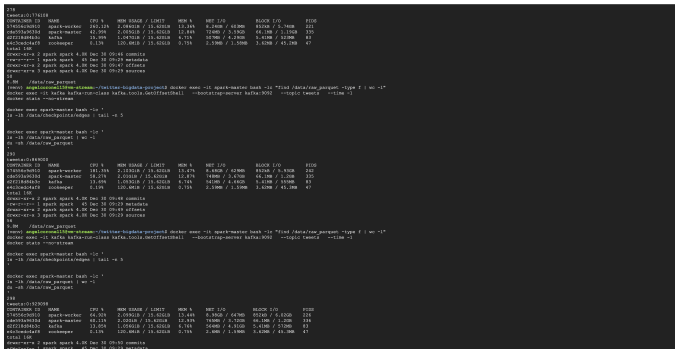


Fig. 9. Statistics with data usage over stress test.

D. Fault Tolerance

To test for fault tolerance, we had to run the producer while the consumer was stopped and rebooted again to see that the system could keep up from the previous saved checkpoint. To do this, the system relies on Kafka to make sure that messages are stored where they need to be and Spark checkpoints track the progress of what was last worked on. Checkpoints are done frequently to make sure any faults don't create a large gap of work already done and work that needs to be redone from the last checkpoint if there is a crash of any sort. This lets us show that the system only processes semantics extacly once without any redundant work.

TABLE IV
CONTAINER-LEVEL RESOURCE USAGE SNAPSHOT

Container	CPU (%)	Memory Usage	Memory (%)	PIDs
spark-worker	0.16	403.8 MiB / 15.62 GiB	2.53	45
spark-master	0.20	298.9 MiB / 15.62 GiB	1.87	46
kafka	1.11	1022 MiB / 15.62 GiB	6.39	82
zookeeper	0.16	120.5 MiB / 15.62 GiB	0.75	47

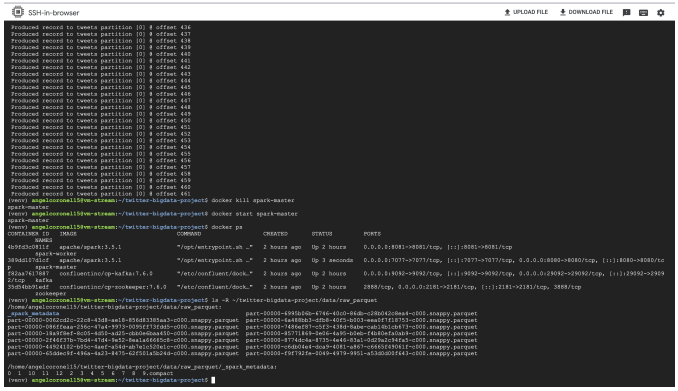


Fig. 10. Testing done with restarting Spark.

VII. LIMITATIONS AND FUTURE WORK

The system was able to show what we seeked out to do. We wanted to show a reliable streaming analytics pipeline for twitter but there are still few limitations left. The scope of the work is limited by the data source and the selected set of

TABLE III
STREAMING STRESS TEST RESULTS

Category	Metric	Observed Value	Interpretation
Workload	Dataset	Sentiment140 (100k tweets replayed)	Realistic social media workload
	Replay rate	$\sim$0.001 s / msg	Highest stable ingestion rate tested
	Run duration	$\sim$25 minutes	Meets "long-running" criterion
Ingestion	Avg inputRowsPerSecond	$\sim$3,200 rows/sec	High sustained input rate
	Peak batch input	$\sim$83,900 rows	Large backlog absorbed
	Kafka lag	0 (avg/max/min)	Consumer kept up
Processing	Avg processedRowsPerSecond	$\sim$4,500 rows/sec	Processing faster than ingestion
	Peak processedRowsPerSecond	$\sim$4,710 rows/sec	Maximum observed throughput
	Processing headroom	$\sim 1.4\times$	System not saturated
Latency	Trigger interval	20 s	Configured micro-batch window
	Max triggerExecution time	$\sim$30 s	Temporary backlog only
	Backpressure warnings	Yes (transient)	Expected under stress
Resources	Spark worker CPU	up to $\sim$221%	Parallel task execution
	Spark worker memory	$\sim$1.3 GB	Stable, no leaks
	Kafka CPU	$<$15%	Kafka not bottleneck
Reliability	Crashes	None	Stable under load
	Checkpoint recovery	Consistent	Fault-tolerant
	Output correctness	Preserved	No data loss
Sink	Output format	Parquet	Efficient analytics storage
	Output growth	Linear	No file explosion

entities that is being kept track of for the relationship graph. Altough its plenty for this project, the dataset used only has a limit of 1.6 million entries and there could be millions more in reality if we were streaming from all over the country or all over the world for certain topics/trends. Also, the entities in the graph don't necessarily reflect the entities that are talked about in the dataset. In the ideal world, the dataset would reflect only what the entities are for using Twitter's API and get relevant tweets. Because this was not monetarily feasible, we had to use the dataset which includes a lot of tweets that have nothing to do with the entities of real interest and acts as noise and gutter within the pipeline. As a result, the outputs show things that are not a part of the initial goal of this project. For future work, a dynamic entities list could be added (not a fixed one) to the system that grows or shrinks based on the stream of tweets and maybe a bias added with what's considered important or not or how much.

The system is also deployed with a limited amount of resources like using a single VM on a small number of containers. Even though the work done is enough for performance, it still doesn't capture what its like at a much larger scale. There could potentially network bottlenecks or machine coordination issues that are not shown in the project. In the same way, the simple sentiment AI tool is a good way to find the sentiment score for a given relationship but its doesn't capture nuance like sarcasm or contextual meaning like memes. For future work, a more nuanced sentiment AI tool could be used for a better judgements.

There are other things that could be done to further improve this project. For example an extra component could be added to the pipeline to support larger clusters and have more Kafka partitions to evaluate horizontal scalability. Possibly training or tuning our own AI sentiment analysis tool would be good for twitter specific texts that includes memes or emojis or twitter trends. A visual component could also be added to the system to do graph analytics or simply visualize how the results are being shown. A dynamic visualization tool native to Apache tools could be used to further add visuals to the project. These things could be added without major changes or having to reconfigure the architecture of the system.

VIII. CONCLUSION

In conclusion, this project showed the design, implementation and evaluation of a real time streaming analytics system for processing big data from social media. With Apache for ingestion, Spark for processing and Parquet for storing, the system transforms unstructured tweets into a structured data of relationship between entities as they are talked about through time. The pipeline shows how big data frameworks can take on unbounded data streams.

Throughout the experiments, the system showed to be very reliable even when stress tested. The resources stayed relatively predictable while the streaming throughput stayed stable and the processing latency consistent. By using checkpointing, we tested and showed fault tolerance keeping correctness. The results are a proof of concept that a compact deployment has the qualities needed for a system used in real world production.

Overall, the project is a good example of applying big data analytics to a real world problem. It shows how batch processing can be used for timely analysis. Also, how unstructured data can provide meaningful insights if processed correctly. The system is a good basis for future work that wants to something similar and cares about scalability, reliability and analysis.

REFERENCES

[1] A. Go, R. Bhayani, and L. Huang, "Sentiment140 dataset with 1.6 million tweets," https://www.kaggle.com/datasets/kazanova/sentiment140, 2009, accessed: Dec. 30, 2025.